

Segment Tree

Presentación por Luciano Agustin Macias



Rango Estático

Los elementos solo son consultados.

No se modifican en ningún momento.

1	2	3	4	5	6	7	8	9	10
2	6	9	1	42	4	7	67	9	8

2	6	9	1
---	---	---	---

Consulta al rango [1 .. 4]

- Sum $\rightarrow$ 18
- Min $\rightarrow$ 1
- Max $\rightarrow$ 9
- XOR $\rightarrow$ 12
- Etc

Rango Dinámico

Se trabaja con consultas y modificaciones de forma arbitraria.

Modificaciones puntuales o en rango.

1	2	3	4	5	6	7	8	9	10
2	6	9	1	42	4	7	67	9	8

2	6	9	1
---	---	---	---

Consulta al rango [1 .. 4]

1	2	3	4	5	6	7	8	9	10
2	6	9	1	42	4	7	67	9	8

1	2	3	4	5	6	7	8	9	10
2	6	9	19	42	4	7	67	9	8

Actualización del elemento 4

Durante la cursada...

→ Sin técnica para rangos estáticos

7	9	4	3	5
---	---	---	---	---



Query $\rightarrow O(n)$

→ Sin técnica para rangos dinámicos

7	9	4	3	5
---	---	---	---	---



Query $\rightarrow O(n)$

Update $\rightarrow O(1)$

Durante la cursada...

→ RSQ para rangos estáticos

7	9	4	3	5
---	---	---	---	---

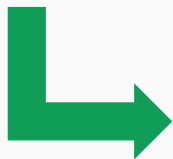


7	$(7+9) = 16$	$(16+4) = 20$	$(20+3) = 23$	$(23+5) = 28$
---	--------------	---------------	---------------	---------------

Query $\rightarrow O(1)$; Update $\rightarrow O(n)$

→ RSQ con actualización "lazy" de los elementos para rangos dinámicos

7	9	4	3	5
---	---	---	---	---



0	2	0	-2	0
---	---	---	----	---

7	$(7+9+2) = 18$	$(18+4+2) = 24$	$(24+3) = 27$	$(27+5) = 32$
---	----------------	-----------------	---------------	---------------

Query $\rightarrow O(1)$; Update $\rightarrow O(n)$ pero se hace solo una vez

Durante la cursada...

→ RSQ para rangos dinámicos

7	9	4	3	5
---	---	---	---	---

7	$(7+9) = 16$	$(16+4) = 20$	$(20+3) = 23$	$(23+5) = 28$
---	--------------	---------------	---------------	---------------

Query $\rightarrow O(1)$; Update $\rightarrow O(n)$

→ Sin técnica para rangos dinámicos

7	9	4	3	5
---	---	---	---	---

Query $\rightarrow O(n)$
Update $\rightarrow O(1)$

Entonces tenemos en rango dinámico:

- Fuerza bruta
 - ◆ Actualización $O(1)$
 - ◆ Consulta $O(n)$
- Algoritmos como Prefix Sums
 - ◆ Actualización $O(n)$
 - ◆ Consulta $O(1)$

Hay que encontrar un punto medio...



Segment Tree

Propiedades de las operaciones

No cualquier operación sirve, la estructura procesa valores de forma arbitraria.

Propiedades de las operaciones soportadas:

→ Asociatividad

$$\forall a, b, c \in Intervalo \rightarrow (a \otimes b) \otimes c = a \otimes (b \otimes c)$$

→ Conmutatividad*

$$\forall a, b, c \in Intervalo \rightarrow (a \otimes b) = (b \otimes a)$$

→ Neutro*

$$\exists e \in U \mid \forall a \in Intervalo \rightarrow (a \otimes e) = (e \otimes a) = a$$

Propiedades de las operaciones

No cualquier operación sirve, la estructura procesa valores de forma arbitraria.

Propiedades de las operaciones soportadas:

→ Asociatividad

$$\forall a, b, c \in \text{Intervalo} \rightarrow (a \otimes b) \otimes c = a \otimes (b \otimes c)$$

→ Conmutatividad*

$$\forall a, b, c \in \text{Intervalo} \rightarrow (a \otimes b) = (b \otimes a)$$

→ Neutro*

$$\exists e \in U \mid \forall a \in \text{Intervalo} \rightarrow (a \otimes e) = (e \otimes a) = a$$

Concepto de Monoide

Propiedades de las operaciones

→ Conmutatividad*

$$\forall a, b, c \in Intervalo \rightarrow (a \otimes b) = (b \otimes a)$$

ó respetar sentido estricto de operación

→ Neutro*

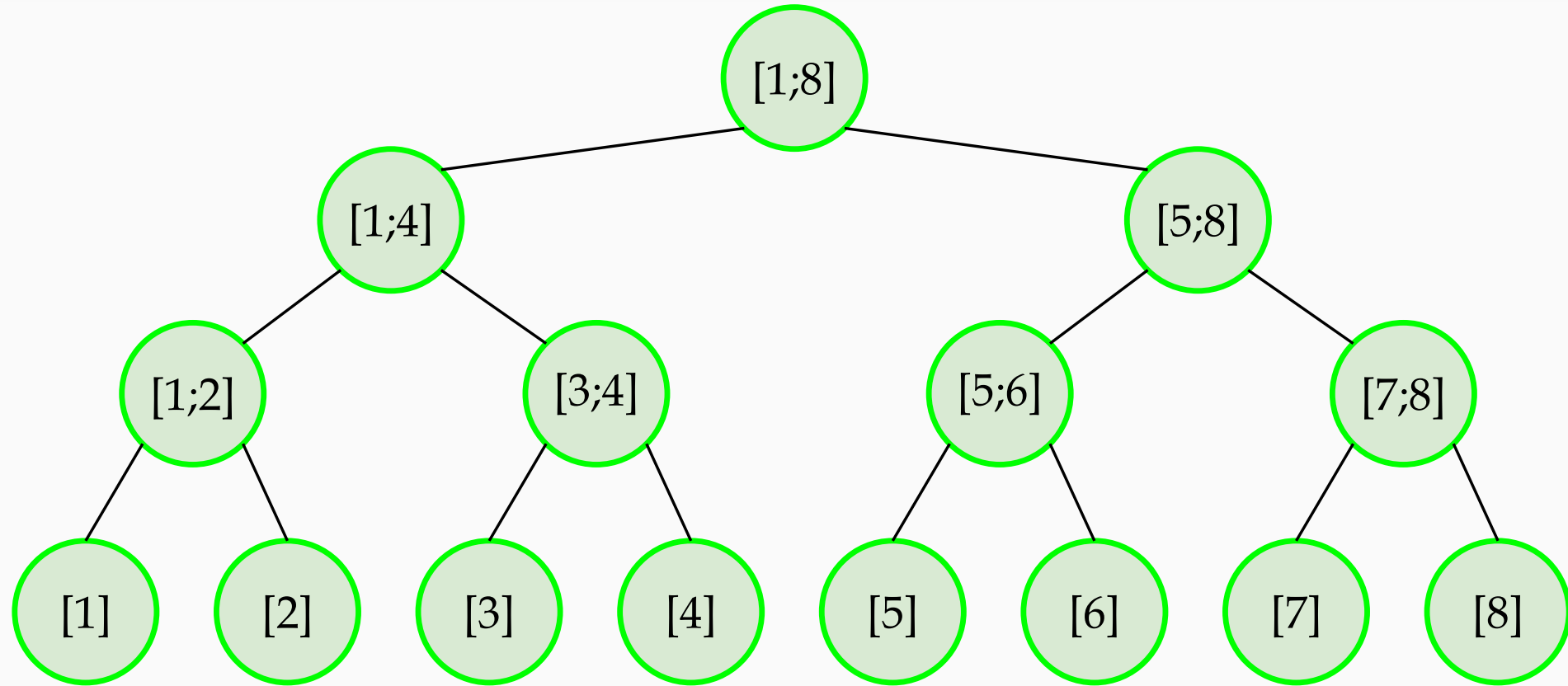
$$\exists e \in U \mid \forall a \in Intervalo \rightarrow (a \otimes e) = (e \otimes a) = a$$

ó controlar casos vacíos / nulos

Ejemplo de operaciones válidas

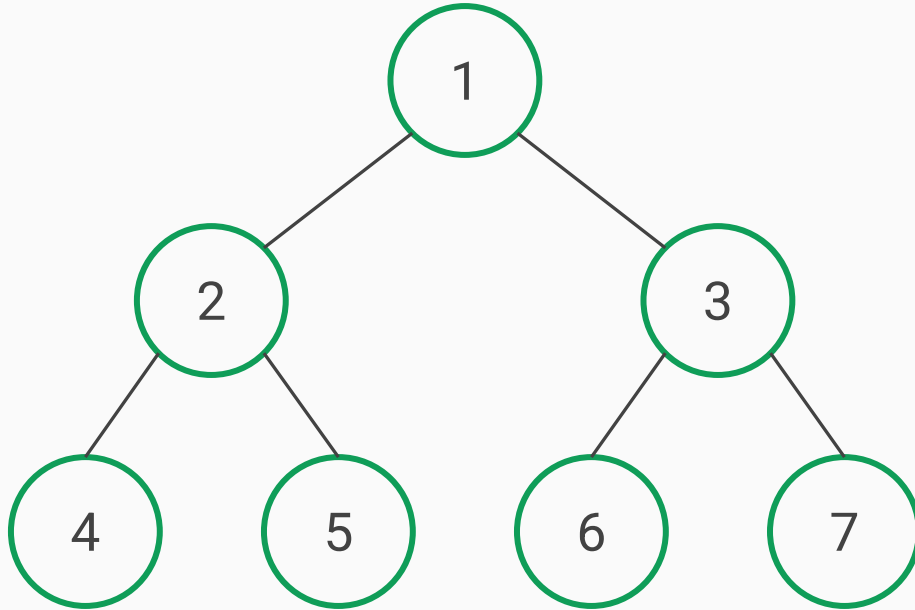
Operación	Neutro
Mínimo	+infinito
Mínimo + Ocurrencias	(+infinito, 0)
Máximo	-infinito
Máximo + Ocurrencias	(-infinito, 0)
Suma	0
Multiplicación	1
XOR	00...0
AND	11...1
OR	00...0
GCD	0

Entre otras como operaciones de matrices, módulo ó bitwise



Estructura y Representación

0	1	2	3	4	5	6	7
-	1	2	3	4	5	6	7

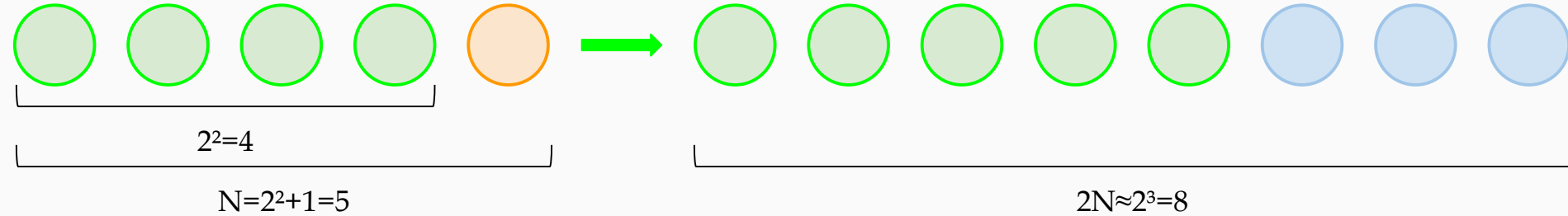


Donde:

- Raíz $\rightarrow$ pos (i)
- HI $\rightarrow$ pos (i*2)
- HD $\rightarrow$ pos (i*2)+1

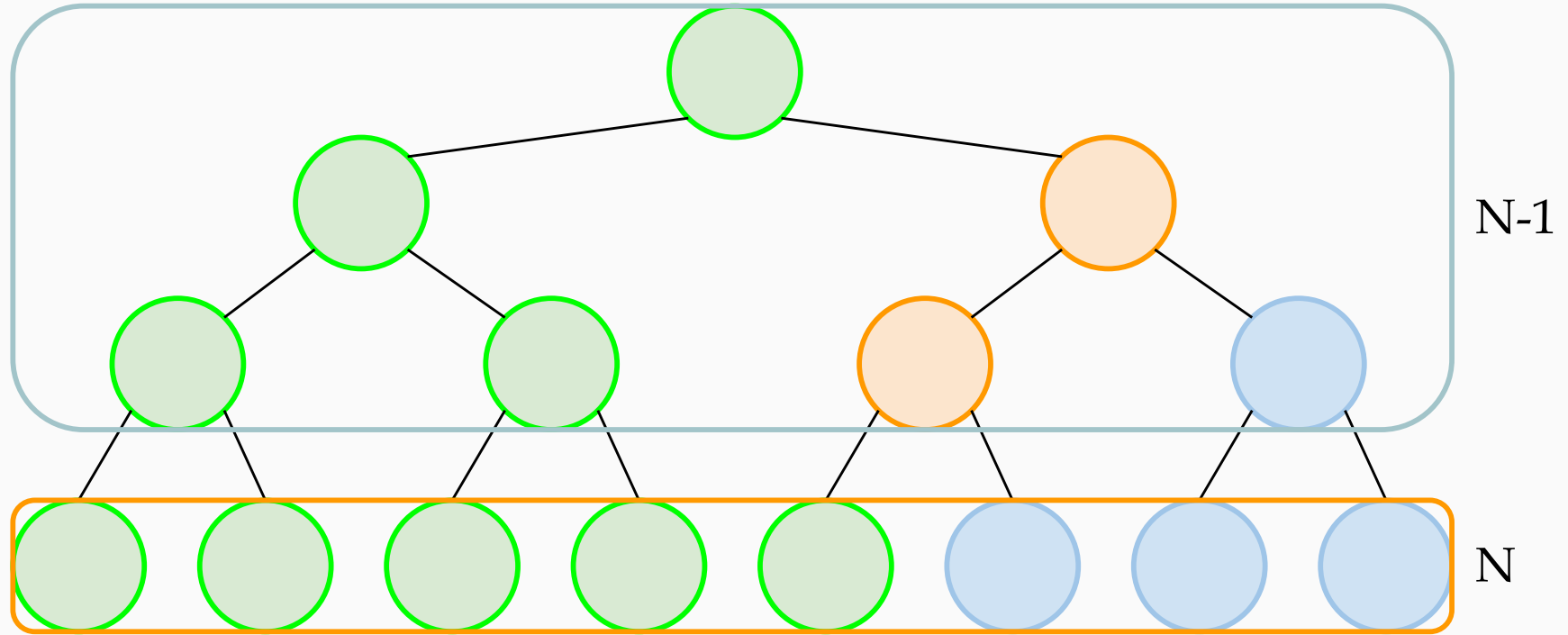
Estructura y Representación

Es necesario que el tamaño del arreglo sea una potencia de 2



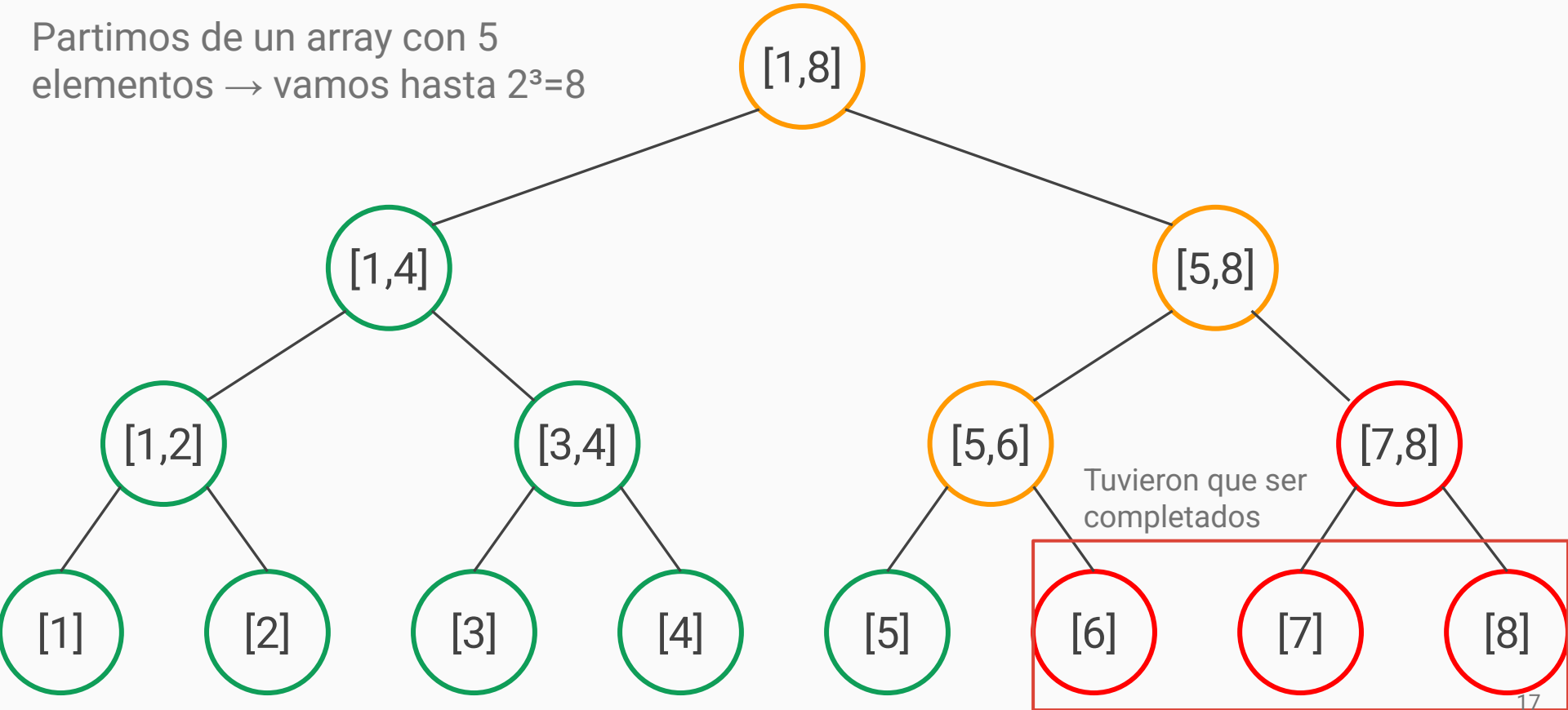
Se completa con valores neutros

Estructura y Representación



Estructura y Representación

Partimos de un array con 5 elementos $\rightarrow$ vamos hasta $2^3=8$



Operaciones del árbol

- $\text{op}(v1, v2) \rightarrow$ Realiza la operación entre dos valores dados
- $\text{build}(\text{idx}, l, r) \rightarrow$ Construye el árbol a partir del arreglo original
- $\text{set}(\text{idx}, \text{val}) \rightarrow$ Establece el valor *val* en la posición *idx* del arreglo
- $\text{query}(l, r) \rightarrow$ Responde a la operación dada en el rango $[L; R-1]$

Operación (op) y Construcción (build)

```
op(v1, v2):  v1 + v2
```

node $\rightarrow$ idx en array del árbol
l y r $\rightarrow$ idx en el array original

```
build(node, l, r):
```

```
    if l == r:
```

```
        tree[node] = arr[l]
```

} Es hoja

```
    return
```

```
    mid = (l + r) / 2
```

```
    build(node*2, l, mid)
```

```
    build(node*2 + 1, mid + 1, r)
```

} Calcula los hijos

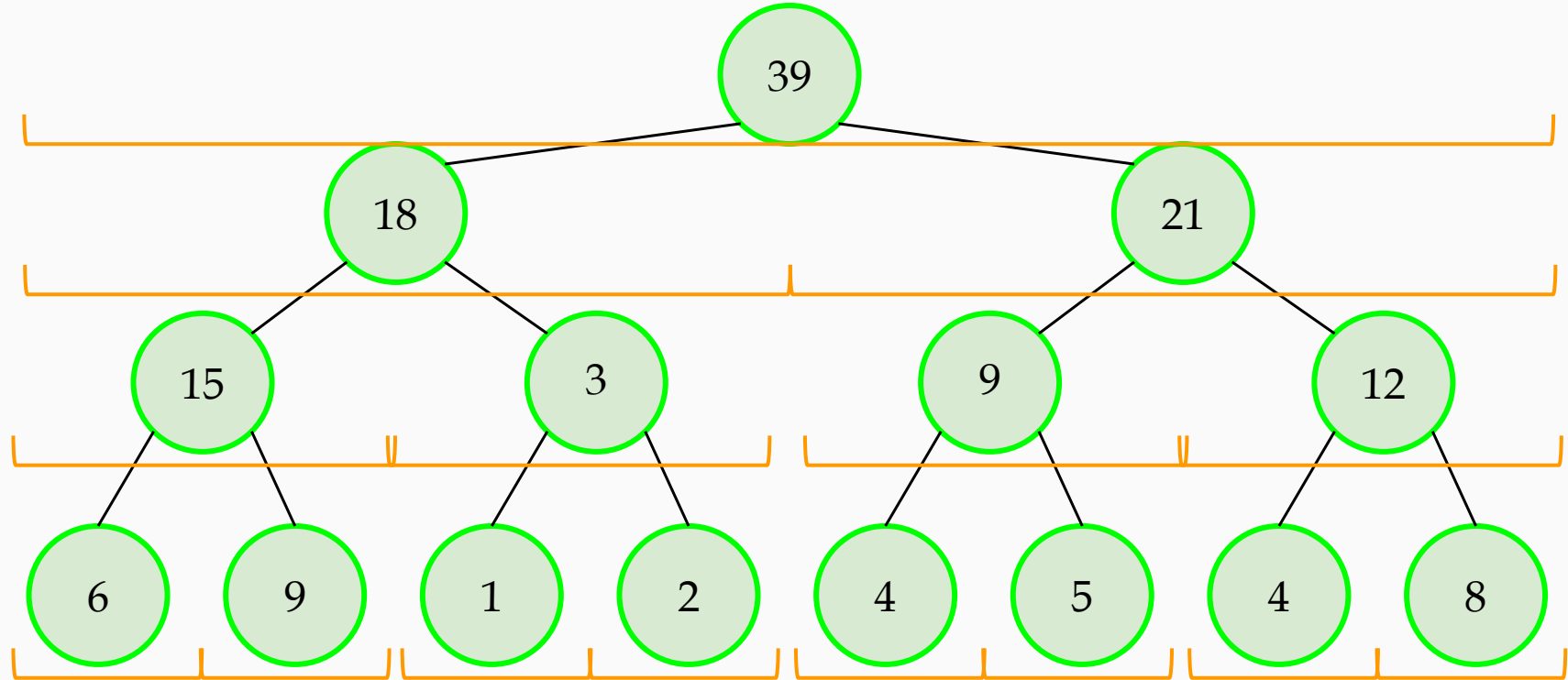
```
    tree[node] = op(tree[node*2], tree[node*2 + 1])
```

} Cálculo propio 19

Construcción (build)

Partimos del arreglo:

6	9	1	2	4	5	4	8
---	---	---	---	---	---	---	---



Actualización (set)

```
set(idx, val): update(1, 1, n, idx, val)
```

```
update(node, l, r, idx, val):
```

```
    if l == r:
        tree[node] = val
    return
```

} Es hoja

```
    mid = (l + r) / 2
```

```
    if idx <= mid:
        update(node*2, l, mid, idx, val)
```

```
    else:
        update(node*2 + 1, mid+1, r, idx, val)
```

```
    tree[node] = op(tree[node*2], tree[node*2 + 1])
```

} Recalcula el nodo

node → idx en array del árbol

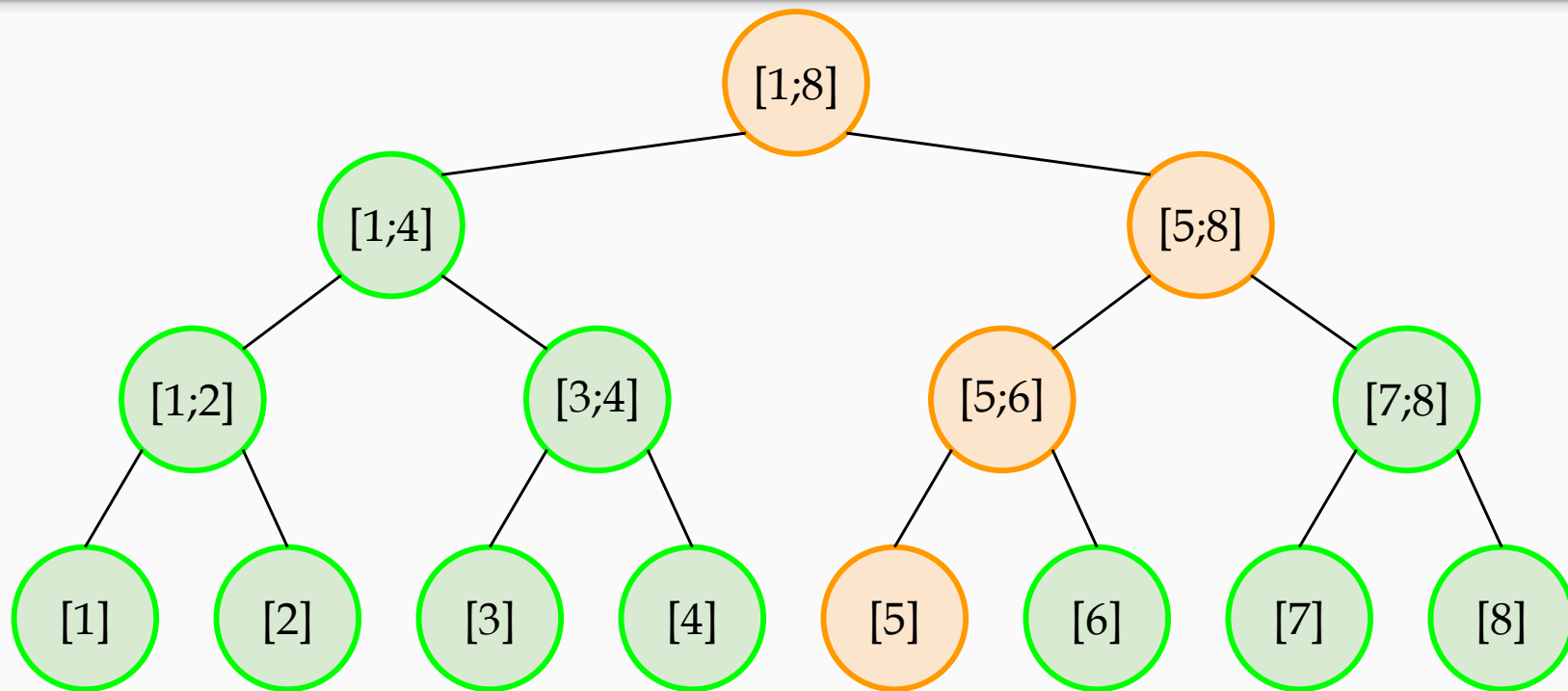
l y r → idx en el array original que ocupa el nodo

idx → idx en array original donde se inserta val

val → valor a insertar

} Busca en el hijo
que corresponde

Actualización (set)



Los nodos **naranja** fueron recalculados

Situaciones:

1. El segmento está fuera del rango buscado.
2. El segmento está completamente contenido en el rango.
3. El segmento se superpone parcialmente con el rango

Consulta (query)

```
query(l, r): return get(1, 1, n, l, r)
```

```
get(node, sl, sr, l, r):
```

```
    if sr < l or r < sl: } Fuera de rango  
        return 0
```

```
    if l <= sl and sr <= r: } Dentro  
        return tree[node]
```

```
    mid = (sl + sr) / 2
```

```
    left = get(node*2, sl, mid, l, r) } Parcialmente
```

```
    right = get(node*2 + 1, mid+1, sr, l, r)
```

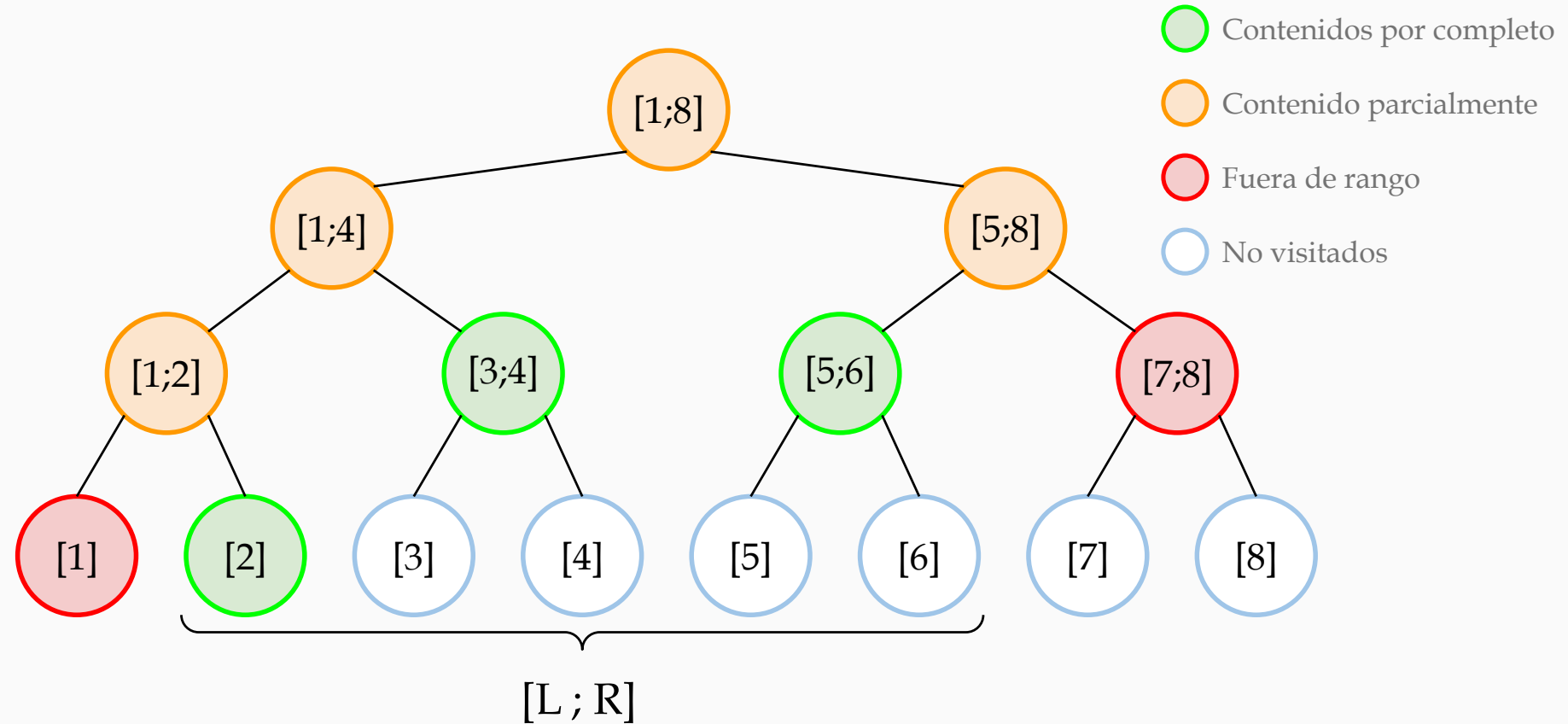
```
    return op(left, right) } Cálculo de partes
```

node → idx en array del árbol

l y r → idx en el array original que se consulta

sl y sr → idx en el array original que corresponden al nodo

Consulta (query)



Segment Tree for the Sum

Input

La primera línea tiene dos enteros n y m ($1 \leq n$ y $m \leq 100000$), siendo n el tamaño del array y m la cantidad de operaciones. La siguiente línea tiene n números que representan el estado inicial del arreglo, cada número puede estar entre $0 \leq a_i \leq 10^9$. Las siguientes m líneas describen las operaciones a realizar, siendo:

- 1 i v : modifica el elemento en el índice i por el valor v ($0 \leq i \leq n$, $0 \leq v \leq 10^9$).
- 2 l r : calcula la suma de los elementos entre los índices l y $r-1$ ($0 \leq l < r \leq n$).

Output

Por cada operación del segundo tipo informar la suma correspondiente.

Código C++ → OP y Build

```
#define ll long long
struct SegTree {
    vector<ll> tree;
    vector<ll> arr;
    // Constructor que llama a build
    ll op(ll a, ll b){ return a + b; }

    void build(ll node, ll l, ll r){
        if (l == r){ tree[node] = arr[l]; return; }
        ll mid = (l + r) / 2;
        build(node * 2, l, mid);
        build(node * 2 + 1, mid + 1, r);
        tree[node] = op(tree[node * 2], tree[node * 2 + 1]);
    }
    // . . .
}
```

Código C++ → Update

```
#define ll long long
struct SegTree {
    // . . .
    void set(ll idx, ll val){ update(1,0,arr.size()-1,idx, val); }

    void update(ll node, ll l, ll r, ll idx, ll val) {
        if (l == r){ tree[node] = val; return; }
        ll mid = (l + r) / 2;
        if (idx <= mid)
            update(node * 2, l, mid, idx, val);
        else
            update(node * 2 + 1, mid + 1, r, idx, val);
        tree[node] = op(tree[node * 2], tree[node * 2 + 1]);
    }
    // . . .
}
```

Código C++ → Query

```
#define ll long long
struct SegTree {
    // . . .
    ll query(ll l, ll r){ return get(1,0,arr.size()-1,l,r); }
    ll get(ll node, ll sl, ll sr, ll l, ll r){
        if (sr < l || r < sl)
            return 0;
        if (l <= sl && sr <= r)
            return tree[node];
        ll mid = (sl + sr) / 2;
        ll left = get(node * 2, sl, mid, l, r);
        ll right = get(node * 2 + 1, mid + 1, sr, l, r);
        return op(left, right);
    }
}
```

Código C++ → Programa Principal

```
int main(){
    FASTIO
    ll n, m;
    cin >> n >> m;
    vector<ll> a(n);
    for (ll i = 0; i < n; i++){
        cin >> a[i];
    }
    SegTree st(a);
    ll comando, p1, p2; // p1= idx o l; p2= val o r
    while(m--){
        cin >> comando >> p1 >> p2;
        if (comando == 1) {
            st.set(p1, p2);
        }else{
            cout << st.query(p1, p2-1) << '\n';
        }
    }
    return 0;
}
```

Submissions

#	When	Who	Problem	Lang	Verdict	Time	Memory
375660397	May/22/2026 12:04 ^{UTC-3}	lordvi	A - Segment Tree for the Sum	C++23 (GCC 14-64, msys2)	Accepted	140 ms	1800 KB

Number of Minimums on a Segment

Input

La primera línea tiene dos enteros n y m ($1 \leq n$ y $m \leq 100000$), siendo n el tamaño del array y m la cantidad de operaciones. La siguiente línea tiene n números que representan el estado inicial del arreglo, cada número puede estar entre $0 \leq a_i \leq 10^9$. Las siguientes m líneas describen las operaciones a realizar, siendo:

- 1 i v : modifica el elemento en el índice i por el valor v ($0 \leq i \leq n$, $0 \leq v \leq 10^9$).
- 2 l r : calcular el mínimo y el número de elementos iguales al mínimo de los elementos entre los índices l y $r-1$ ($0 \leq l < r \leq n$).

Output

Por cada operación del segundo tipo informar dos enteros: el mínimo del segmento y el número de elementos iguales al mínimo.

Código C++ → Estructura Mínimo

```
#define ll long long
const ll INF = 1000000000LL;

struct Minimo
{
    ll minimo = INF;
    ll ocurrencias = 0;
    Minimo(ll min = INF, ll ocu = 0)
        : minimo(min), ocurrencias(ocu){}
};
```


Código C++ → Cambio en operación

```
#define ll long long                                const ll INF = 1000000000LL;
struct SegTree {
    vector<Minimo> tree; // PRIMER CAMBIO
    vector<ll> arr;
    // Constructor que llama a build
    // OP cambia para efectuar la operación pedida
    Minimo op(Minimo a, Minimo b){
        if(a.minimo==b.minimo){
            return Minimo(a.minimo, a.ocurrencias+b.ocurrencias);
        }
        return (a.minimo<b.minimo) ? a : b;
    }
    // . . .
}
```

Código C++ → Build

```
#define ll long long                                const ll INF = 1000000000LL;
struct SegTree {
    // . . .
    void build(ll node, ll l, ll r){
        if (l == r){
            tree[node] = Minimo(arr[l], 1); // TERCER CAMBIO
            return;
        }
        ll mid = (l + r) / 2;
        build(node * 2, l, mid);
        build(node * 2 + 1, mid + 1, r);
        tree[node] = op(tree[node * 2], tree[node * 2 + 1]);
    }
    // . . .
}
```

Código C++ → Update

```
#define ll long long                                const ll INF = 1000000000LL;
struct SegTree {
    // . . .
    void set(ll idx, ll val){ update(1,0,arr.size()-1,idx, val); }
    void update(ll node, ll l, ll r, ll idx, ll val) {
        if (l == r){
            tree[node] = Minimo(val, 1); // CUARTO CAMBIO
            return;
        }
        ll mid = (l + r) / 2;
        if (idx <= mid) update(node * 2, l, mid, idx, val);
        else update(node * 2 + 1, mid + 1, r, idx, val);
        tree[node] = op(tree[node * 2], tree[node * 2 + 1]);
    }
    // . . .
}
```

Código C++ → Query

```
#define ll long long                                const ll INF = 1000000000LL;
struct SegTree {
    // . . .
    Minimo query(ll l, ll r){return get(1,0,arr.size()-1,l,r);}
    Minimo get(ll node, ll sl, ll sr, ll l, ll r)
    {
        if (sr < l || r < sl)
            return Minimo(); // 5TO CAMBIO
        if (l <= sl && sr <= r)
            return tree[node];
        ll mid = (sl + sr) / 2;
        Minimo left = get(node * 2, sl, mid, l, r); // 6TO
        Minimo right = get(node * 2 + 1, mid + 1, sr, l, r); //7MO
        return op(left, right);
    }
}
```

Código C++ → Programa Principal

```
int main(){
    FASTIO
    ll n, m;
    cin >> n >> m;
    vector<ll> a(n);
    for (ll i = 0; i < n; i++){ cin >> a[i]; }
    SegTree st(a);
    ll comando, p1, p2; // p1= idx o l; p2= val o r
    while(m--){
        cin >> comando >> p1 >> p2;
        if (comando == 1) {
            st.set(p1, p2);
        }else{
            // 8VO CAMBIO
            Minimo m = st.query(p1, p2-1);
            cout << m.minimo << ' ' << m.ocurrencias << '\n';
        }
    }
    return 0;
}
```

Submissions

#	When	Who	Problem	Lang	Verdict	Time	Memory
375662699	May/22/2026 12:25 UTC-3	lordvi	C - Number of Minimums on a Segment	C++23 (GCC 14-64, msys2)	Accepted	140 ms	5000 KB

Extensión y Mejoras

Como todas las estructuras, existen mejoras que permiten expandir el horizonte de problemas.

→ Actualización eficiente en rango

De $N \log(N) \rightarrow \log(N)$

→ Actualización diferida (Lazy Propagation)

*Se registra la **actualización pendiente** y solo se efectúa si se consulta o actualiza nuevamente el rango. **Ahorra recursos en actualizaciones que quizás nunca se usen.***

Aplicaciones más allá de la Programación Competitiva

- **Renderizado de gráficos** → algoritmos de trazado de rayos.
- **Occlusion culling** → algoritmos para evitar renderizar objetos que no son visibles.
- **Geometría computacional y GIS** → problema de la ventana para un conjunto de líneas

Conclusión

Si bien es la “navaja suiza de los problemas de rango dinámico”, hay que saber elegir cuándo y dónde utilizarlo

- Fundamentos matemáticos sólidos
- Código base altamente reutilizable
- Versatilidad en operaciones soportadas

Muchas Gracias
¿Preguntas?